

Projecting Radiance Fields to Mesh Surfaces

Adrian Xuan Wei Lim
Roblox
USA
xlim@roblox.com

Lynnette Hui Xian Ng
Carnegie Mellon
University
USA
lynnetteng@cmu.edu

Nicholas Kyger
Roblox
USA
nkyger@roblox.com

Tomo Michigami
Roblox
USA
tmichigami@roblox.com

Faraz Baghernezhad
Roblox
USA
fbaghernezhad@roblox.com

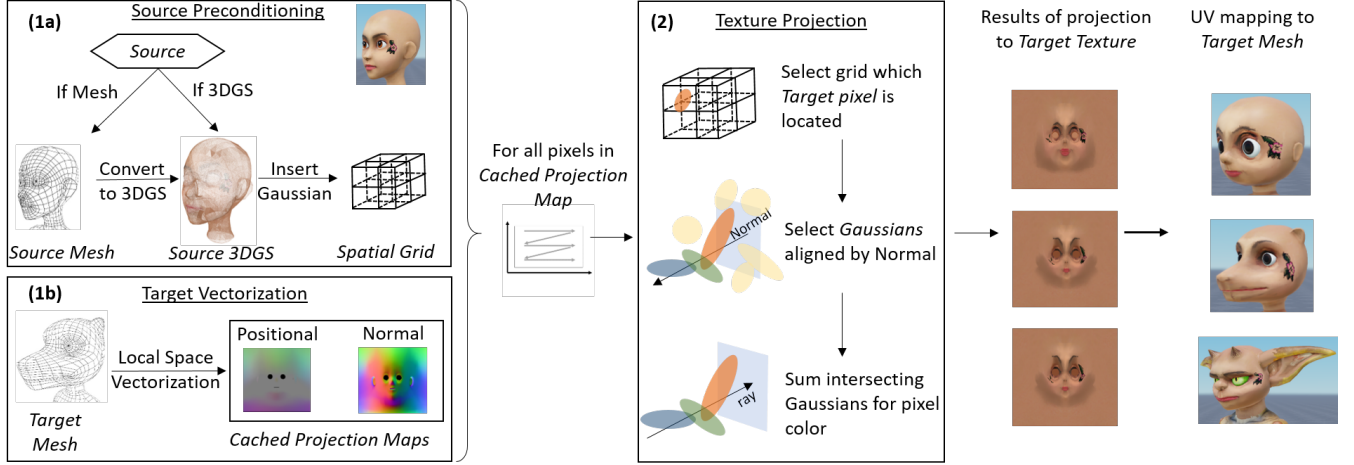


Figure 1: Projection Pipeline, with example of texture transfer from one source avatar to three differently shaped target meshes.

ABSTRACT

Radiance fields produce high fidelity images with high rendering speed, but are difficult to manipulate. We effectively perform avatar texture transfer across different appearances by combining benefits from radiance fields and mesh surfaces. We represent the source as a radiance field using 3D Gaussian Splatting, then project the Gaussians on the target mesh. Our pipeline consists of Source Preconditioning, Target Vectorization and Texture Projection. The projection completes in 1.12s in a pure CPU compute, compared to baselines techniques of Per Face Texture Projection and Ray Casting (31s, 4.1min). This method lowers the computational requirements, which makes it applicable to a broader range of devices from low-end mobiles to high end computers.

ACM Reference Format:

Adrian Xuan Wei Lim, Lynnette Hui Xian Ng, Nicholas Kyger, Tomo Michigami, and Faraz Baghernezhad. 2024. Projecting Radiance Fields to Mesh Surfaces. In *Special Interest Group on Computer Graphics and Interactive Techniques Conference Posters (SIGGRAPH Posters '24)*, July 27–August 01, 2024, Denver, CO, USA. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3641234.3671036>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

SIGGRAPH Posters '24, July 27–August 01, 2024, Denver, CO, USA

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0516-8/24/07

<https://doi.org/10.1145/3641234.3671036>

1 INTRODUCTION

Texture transfer between differently shaped avatars is an important rendering problem. Work on texture transfer includes mesh methods such as using adaptive triangulations of meshes, which completes 11 texture transfers in 38 minutes [Schmidt et al. 2023]; or mapping pixels from the UV parameterization of a surface to the 3D avatar which was used to transfer clothing onto avatars [Mir et al. 2020]; or deep learning methods that perform direct image-to-image translation [Shysheya et al. 2019]. However, these methods take a long time to process and render the textures.

To speed up the texture transfer process, radiance field rendering methods using 3D Gaussians (3DG) can be harnessed. 3DG can be used to preserve the continuous properties of the original texture, and avoid unnecessary computation in empty spaces [Kerbl et al. 2023]. Most importantly, such a technique preserves rendering quality. Kerbl et al. [2023] used 3DG as volumetric representation for real-time scene rendering. Wang and Sin [2024] generated 3DG Textures for volumetric texturing in animations.

In this work, we present a method for avatar texture transfer. The texture transfer is mediated through a Gaussian splatting-based pipeline. We represent our source radiance field as a 3D Gaussian Splatting, and incorporated local space projection Lim et al. [2023] to cache and speed up locality projection. We then directly project the Gaussians on the target texture, blending them for colour and lighting. Our technique efficiently completes realistic avatar texture transfer across different appearances in 1.12s.

2 METHODOLOGY

Figure 1 shows the Projection Pipeline and the resultant outputs for three target shapes from a single source mesh. The figure illustrates transferring face tattoo and makeups from the face of one avatar to three different meshes of different shapes.

Source Preconditioning. We accept two source types: (1) 3D polygon mesh, and (2) a 3DGS. To convert a 3D mesh into 3DGS, we generated a Gaussian per pixel of its texture. The Gaussian's position and normal corresponds to the current pixel local space. The covariance spread is determined by its rate of change of its positional iteration. The colour corresponds to the current pixel colour. We then densify the 3DGS to have uniform surface density and to increase the overlap between Gaussians.

We construct a Spatial Grid to contain ~ 1 target pixel per subgrid. The grid size is cube root of total target pixel count. The Gaussians are added into all subgrids of the Spatial Grid that collides with their covariance volume. This grid setup converts a polynomial to constant search at projection time by caching pixel locality information during creation, eliminating the need for a global search for neighbouring Gaussians for each pixel during projection.

Target Vectorization. We rasterize all triangles in the target mesh to their UV-space, converting the mesh into Cached Projection Maps. Those texture maps contains the local space position and normal for all target pixels. There is a one-to-one size match between our Cached Projection Maps and our target texture to allow for constant space and time lookups.

Texture Projection. For each pixel in the Cached Projection Map, we index the subgrid based on its position. Next, we filter for Gaussians in the indexed subgrid that are aligned with the pixel with a dot product between each Gaussian and the normal. We then perform a ray cast from the target position and normal. The colours of the Gaussians intersecting the ray are blended by their alpha values, and are weighted by their angle to the normal, forming the colour of the target texture pixel.

3 RESULTS AND DISCUSSION

Performance. Our experiments used a single-thread CPU (Intel i9-12900H) implementation. Given a mesh with $\sim 10k$ triangles, $\sim 1m$ Gaussians and a texture size of 1024×1024 , our pipeline completed in $1.12 \pm 0.05s$. This timing result was recorded from differing source and targets with similar specifications.

The pipeline can be sped up further by parallelizing the following steps: Spatial Grid insertion, Local Space Vectorization, and the Target Projection. Experimenting with thread counts of 2 and 4, the pipeline completes in 0.68s and 0.46s respectively. Therefore, using parallelization, we achieve near linear decrease in processing time.

Our pipeline is much faster compared to baseline methods on the same CPU: 31sec for Per Face Texture Projection and 4.1min for Ray Cast technique. Rendering quality is not sacrificed in the speed gain, illustrated by the examples in Figure 1 which presents high fidelity images.

Accuracy. We ran a test on 5 different mesh-based avatar heads. We projected these avatar heads onto themselves to extract the transferred texture. We then compared the result of the transferred

texture with the original texture. The two textures are $\sim 98\%$ similar. The errors could be introduced during inaccuracies propagated in the 3DGS densification step.

Deployability. Our test GPU implementation achieved real-time performance but low-end mobile GPUs do not support rasterization to floating point textures, thus unable to compute Cached Projection Maps. However, a pure CPU approach allows deployment to low-end devices, opening possibilities to leverage on existing compute cloud infrastructure.

Limitations. Texture transfer to targets that have different mesh shapes from the source will result in a distortion of projection. This is similar to artifacts generated during a planar projection on a sphere. This can be resolved by enclosing the target in a cage and applying a warp deformer to closely match the shape of the target to the source.

In a mirrored or tiled target, texture transfer with our pipeline can be duplicated or overridden depending on the order of execution of the transfer. A possible resolution to this is to UV-unwrap the target mesh to be UV-space unique.

Future Work. Future work involves object masking for the removal of facial accessories, so that only textures are mapped over, and content-aware fill for predicting eye colors during projection. These enhancements will create more photorealistic avatars.

Conclusions. Our projection pipeline efficiently transfers textures across source-texture pairs of different shapes by combining the high fidelity benefits of radiance fields with the manipulability of polygon meshes, opening possibilities to animate the target model.

ACKNOWLEDGMENTS

Much thanks to Charlie Kubal, Jovanni Cutigni, Morgan McGuire, Victor Zordan, and the amazing folks at Roblox and CMU for their support in this research. Special thanks to Roblox's Game Engine Group, Avatar Personalization Team, Rendering team for support in implementation.

REFERENCES

- Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 2023. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. arXiv:2308.04079 [cs.GR]
- Adrian Xuan Wei Lim, Lynnette Hui Xian Ng, Conor Griffin, Nicholas Kryer, and Faraz Baghernezhad. 2023. Reverse Projection: Real-Time Local Space Texture Mapping. In *ACM SIGGRAPH 2023 Posters* (Los Angeles, CA, USA) (SIGGRAPH '23). Association for Computing Machinery, New York, NY, USA, Article 48, 2 pages. <https://doi.org/10.1145/3588028.3603653>
- Aymen Mir, Thiemo Alldieck, and Gerard Pons-Moll. 2020. Learning to transfer texture from clothing images to 3d humans. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 7023–7034.
- Patrick Schmidt, Dörte Pieper, and Leif Kobbelt. 2023. Surface Maps via Adaptive Triangulations. *Computer Graphics Forum* 42, 2 (2023).
- Aliaksandra Shysheya, Egor Zakharov, Kara-Ali Aliiev, Renat Bashirov, Egor Burkov, Karim Isakov, Aleksei Ivakhnenko, Yury Malkov, Igor Pasechnik, Dmitry Ulyanov, et al. 2019. Textured neural avatars. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2387–2397.
- Xiangzhi Eric Wang and Zackary P. T. Sin. 2024. 3D Gaussian Model for Animation and Texturing. arXiv:2402.19441 [cs.GR]